

Cross-Entropy and KL Divergence

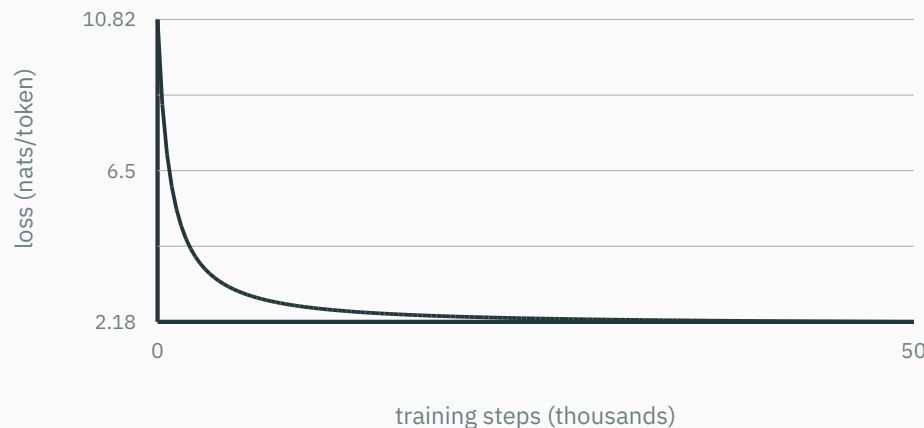
The cost of using the wrong probability model

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

**A code built for the wrong
weather**

The loss curve every language model reports



The training curve of a GPT-style model plots **cross-entropy**: the average of $-\ln q(\text{next token})$. At initialization the model is near uniform over its 50257-token vocabulary, so the loss starts near $\ln 50257 \approx 10.8$ nats.

This lecture defines that loss, proves that minimizing it is maximum likelihood, and returns to read this curve's final value.

True weather and forecast model

Suppose the actual distribution is

$$p = (0.75, 0.20, 0.05)$$

for sun, cloud, and rain.

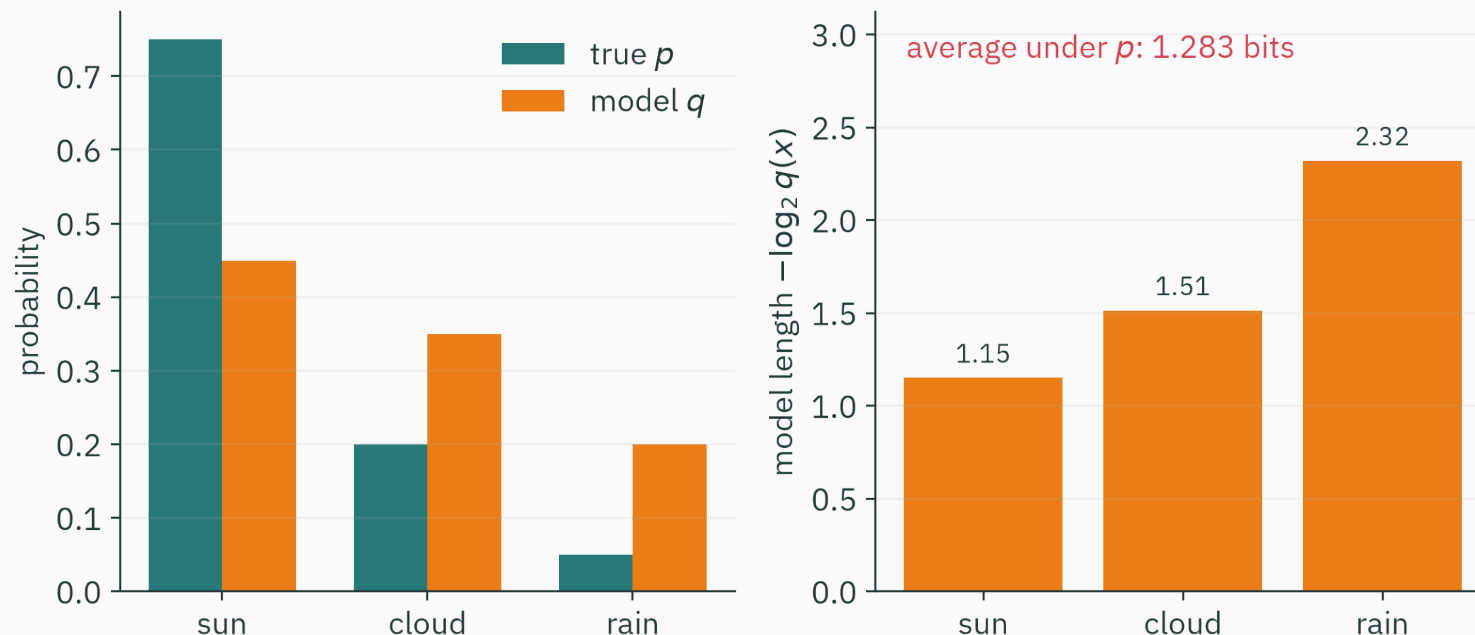
A forecasting model reports

$$q = (0.45, 0.35, 0.20).$$

If code lengths are built from q , outcome x receives ideal length

$$-\log_2 q(x).$$

Model probabilities determine lengths



Sun's codeword costs $-\log_2 0.45 \approx 1.15$ bits; rain's costs $-\log_2 0.20 \approx 2.32$ bits.

The average uses true frequencies p , but the lengths come from model probabilities q .

Average cost under the true source

Define cross-entropy

$$H(p, q) = - \sum_x p(x) \log_2 q(x).$$

For this weather example,

$$H(p, q) = -0.75 \log_2 0.45 - 0.20 \log_2 0.35 - 0.05 \log_2 0.20.$$

$$H(p, q) \approx 1.283 \text{ bits/outcome.}$$

Compare with the true entropy

L22 computed

$$H(p) = - \sum_x p(x) \log_2 p(x) \approx 0.992 \text{ bits/outcome.}$$

Using q instead of p costs

$$1.283 - 0.992 = 0.291 \text{ extra bits/outcome.}$$

That excess is the KL divergence from p to q .

Learning outcomes

By the end of this lecture you will be able to:

1. compute cross-entropy and KL divergence for discrete distributions,
2. derive ★ $H(p, q) = H(p) + D_{\text{KL}}(p \| q)$,
3. prove KL non-negativity using Jensen's inequality,
4. explain KL asymmetry and support failures,
5. show that minimizing cross-entropy is maximum likelihood,
6. reduce one-hot classification cross-entropy to $-\log q_y$,
7. convert average log loss to perplexity,
8. and interpret mutual information as dependence.

Entropy plus an overcharge

Three related quantities

Entropy of the true source:

$$H(p) = - \sum_x p(x) \log p(x).$$

Cross-entropy of model q under source p :

$$H(p, q) = - \sum_x p(x) \log q(x).$$

KL divergence:

$$D_{\text{KL}(p\|q)} = \sum_x p(x) \log \left(\frac{p(x)}{q(x)} \right).$$

★ Derive the identity

Start from cross-entropy and multiply inside the logarithm by $\frac{p(x)}{p(x)}$:

$$H(p, q) = - \sum_x p(x) \log q(x).$$

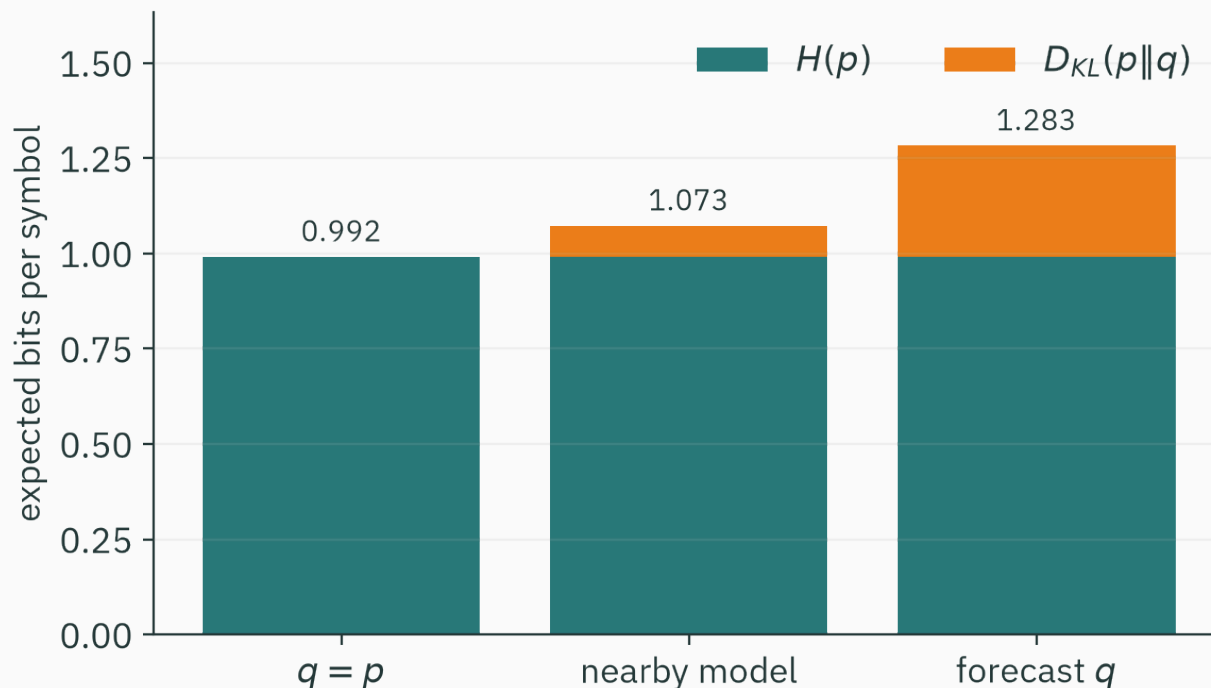
$$= - \sum_x p(x) \log p(x) + \sum_x p(x) \log \left(\frac{p(x)}{q(x)} \right).$$

$$H(p, q) = H(p) + D_{\text{KL}}(p \| q).$$

Read each term

$$\underbrace{H(p, q)}_{\text{actual average length using model } q}$$
$$= \underbrace{H(p)}_{\text{unavoidable source uncertainty}}$$
$$+ \underbrace{D_{\text{KL}}(p \| q)}_{\text{extra cost from mismatch}} .$$

For a fixed source p , only the KL term depends on q .



The teal band $H(p) \approx 0.992$ bits never moves. The forecast q from the opening pays 1.283 bits; the entire difference is the orange mismatch term.

Weather KL calculation

$$D_{\text{KL}(p\|q)} = \sum_x p(x) \log_2 \left(\frac{p(x)}{q(x)} \right).$$

$$= 0.75 \log_2 \left(\frac{0.75}{0.45} \right) + 0.20 \log_2 \left(\frac{0.20}{0.35} \right) + 0.05 \log_2 \left(\frac{0.05}{0.20} \right).$$

$$\approx 0.291 \text{ bits.}$$

Some individual terms are negative; their weighted sum is never negative.

The machine agrees · three sums, one identity

```
import numpy as np
p = np.array([0.75, 0.20, 0.05])      # true weather
q = np.array([0.45, 0.35, 0.20])      # forecast model
H = -(p * np.log2(p)).sum()           # entropy
CE = -(p * np.log2(q)).sum()          # cross-entropy
KL = (p * np.log2(p / q)).sum()       # KL divergence
print(f"{H:.3f} {CE:.3f} {KL:.3f}")   # 0.992 1.283 0.291
print(np.isclose(CE, H + KL))         # True
```

The decomposition holds to machine precision, not only on the slide.

If $q = p$, what are $D_{\text{KL}}(p\|q)$ and $H(p, q)$?

A. 0 and 0

B. 0 and $H(p)$

C. $H(p)$ and 0

D. $H(p)$ and $2H(p)$

Correct: $B - 0$ and $H(p)$

Why: A perfect model removes the mismatch cost, but it does not remove the source's intrinsic entropy.

A divergence, not a distance

KL is non-negative

For distributions p and q on the same outcome set,

$$D_{\text{KL}}(p\|q) \geq 0,$$

with equality if and only if $p = q$ almost everywhere.

This makes cross-entropy no smaller than entropy:

$$H(p, q) \geq H(p).$$

If $q(x) = 0$ for an outcome with $p(x) > 0$, then $D_{\text{KL}(p\|q)} = \infty$ and the claim already holds.

Otherwise let S be the support of p . Since $-\log z$ is convex,

$$D_{\text{KL}(p\|q)} = - \sum_{x \in S} p(x) \log \left(\frac{q(x)}{p(x)} \right).$$

Jensen's inequality gives

$$\begin{aligned} &\geq -\log \left(\sum_{x \in S} p(x) \frac{q(x)}{p(x)} \right) \\ &= -\log \left(\sum_{x \in S} q(x) \right) \geq -\log 1 = 0. \end{aligned}$$

★ Equality conditions

D DERIVATION

Equality requires $\frac{q(x)}{p(x)}$ to be constant on S and q to put no mass outside S .
Normalization then makes that constant one, so $q = p$.

KL is asymmetric

Usually

$$D_{\text{KL}}(p\|q) \neq D_{\text{KL}}(q\|p).$$

Example:

$$p = (0.9, 0.1), \quad q = (0.5, 0.5).$$

$$D_{\text{KL}}(p\|q) \approx 0.531 \text{ bits},$$

$$D_{\text{KL}}(q\|p) \approx 0.737 \text{ bits}.$$

Swapping the arguments changes which distribution supplies the average.

KL does not obey the triangle inequality

A metric distance d must be symmetric and satisfy

$$d(p, r) \leq d(p, q) + d(q, r).$$

KL is asymmetric and does not generally satisfy this triangle inequality.

Therefore call it a **divergence**, not a distance.

Missing support gives infinite cost

If $p(x) > 0$ but $q(x) = 0$, then

$$-\log q(x) = \infty$$

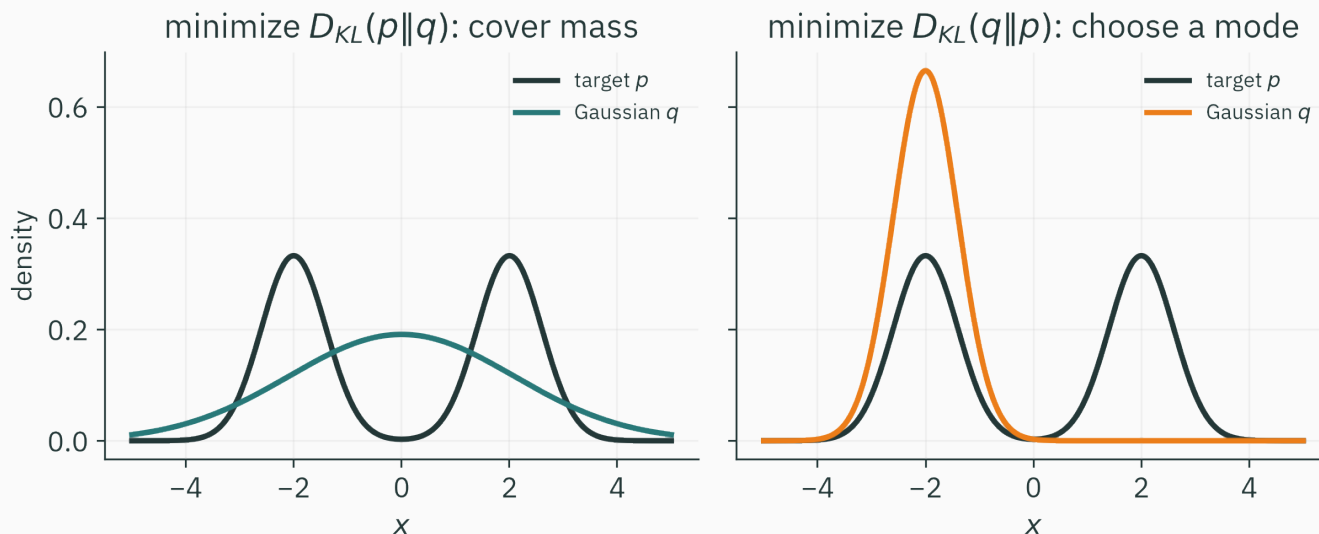
and

$$D_{\text{KL}}(p\|q) = \infty.$$

A model that declares a possible observation impossible receives infinite ideal log loss.

This is one reason probability smoothing matters.

Two directions, two failure modes



Fit one Gaussian q to a bimodal target p . Minimizing the **forward** direction $D_{KL}(p\|q)$ averages $\log(p/q)$ under p , so q must put probability wherever p has mass — the fit is **mass-covering**.

Minimizing the **reverse** direction $D_{KL}(q\|p)$ averages under q , so q avoids regions where $p \approx 0$ and settles on one mode — **mode-seeking**, also called zero-forcing.

Which direction appears in supervised learning?

Data are drawn from an unknown source p . A model supplies q_θ .

Expected log loss is

$$\mathbb{E}_{X \sim p}[-\log q_\theta(X)] = H(p, q_\theta).$$

Therefore the relevant mismatch is

$$D_{\text{KL}}(p \| q_\theta).$$

The source distribution supplies the average: training on log loss minimizes the forward, mass-covering direction.

Cross-entropy is negative log-likelihood

Dataset negative log-likelihood

For independent observations $x_1, \dots, x_n$ under model q_θ ,

$$L(\theta) = \prod_{i=1}^n q_\theta(x_i).$$

Negative log-likelihood is

$$\text{NLL}(\theta) = - \sum_{i=1}^n \log q_\theta(x_i).$$

Divide by n to compare datasets of different sizes.

Group observations by symbol

Let n_x be the count of symbol x , and $\hat{p}(x) = \frac{n_x}{n}$.

$$\frac{1}{n} \text{NLL}(\theta) = -\frac{1}{n} \sum_x n_x \log q_\theta(x).$$

$$= -\sum_x \hat{p}(x) \log q_\theta(x).$$

$$\frac{1}{n} \text{NLL}(\theta) = H(\hat{p}, q_\theta).$$

The complete equivalence

$$\arg \min_{\theta} H(\hat{p}, q_{\theta})$$

$$= \arg \min_{\theta} [H(\hat{p}) + D_{\text{KL}}(\hat{p} \| q_{\theta})]$$

$$= \arg \min_{\theta} D_{\text{KL}}(\hat{p} \| q_{\theta})$$

$$= \arg \max_{\theta} \sum_i \log q_{\theta}(x_i).$$

The empirical entropy is constant with respect to model parameters.

Bernoulli example: eight heads, two tails

The empirical distribution is

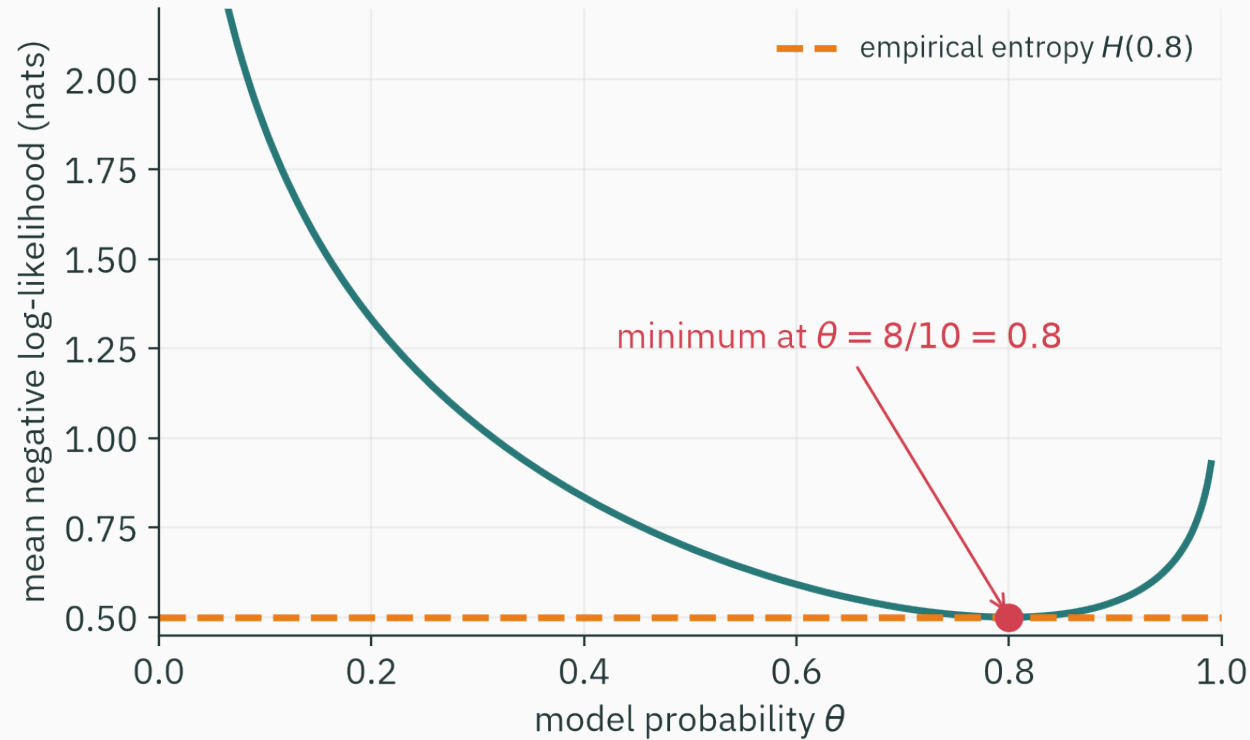
$$\hat{p} = (0.8, 0.2).$$

A Bernoulli model uses

$$q_{\theta} = (\theta, 1 - \theta).$$

Mean NLL, or binary cross-entropy, is

$$-0.8 \ln \theta - 0.2 \ln(1 - \theta).$$



The minimum occurs at $\theta = 0.8$, exactly the Bernoulli MLE from L14.

Differentiate the Bernoulli loss

$$\mathcal{L}(\theta) = -0.8 \ln \theta - 0.2 \ln(1 - \theta).$$

$$\mathcal{L}'(\theta) = -\frac{0.8}{\theta} + \frac{0.2}{1 - \theta}.$$

Set the derivative to zero:

$$0.2\theta = 0.8(1 - \theta) \quad \Rightarrow \quad \theta = 0.8.$$

The second derivative is positive on $(0, 1)$, so this is the global minimum.

Categorical cross-entropy

One-hot target

For K classes, let target vector $\mathbf{y}$ be one-hot and prediction $\mathbf{q}$ be a probability vector.

$$\text{CE}(\mathbf{y}, \mathbf{q}) = - \sum_{k=1}^K y_k \log q_k.$$

If the correct class is c , then only $y_c = 1$:

$$\text{CE} = -\log q_c.$$

Numerical example

Correct class: cat.

model	$q(\text{cat})$	loss $-\ln q(\text{cat})$
A	0.90	0.105
B	0.60	0.511
C	0.10	2.303

Confident correct predictions receive small loss; confident errors receive large loss.

Soft labels

If the target is a distribution p rather than one-hot,

$$\text{CE}(p, q) = - \sum_k p_k \log q_k.$$

This occurs with:

1. label smoothing,
2. distillation from a teacher model,
3. probabilistic annotations,
4. mixture targets.

The same decomposition $H(p, q) = H(p) + D_{\text{KL}}(p \| q)$ still applies.

Softmax and numerical stability

Logits z_k produce

$$q_k = \frac{\exp z_k}{\sum_j \exp z_j}.$$

Compute log probabilities using log-sum-exp:

$$\log q_k = z_k - \text{logsumexp}(\mathbf{z}).$$

This avoids overflow and underflow from L2. Practical libraries combine `log_softmax` with NLL loss.

F.cross_entropy verified

```
import torch, torch.nn.functional as F
logits = torch.tensor([[2.0, -1.0, 0.5]]) # raw scores, 3 classes
y = torch.tensor([0]) # correct class: index 0
q = F.softmax(logits, dim=1) # (0.7856, 0.0391, 0.1753)
print(-torch.log(q[0, 0])) # tensor(0.2413)
print(F.cross_entropy(logits, y)) # tensor(0.2413)
```

F.cross_entropy takes raw logits and computes $-\log q_c$ with log-sum-exp inside — L14 named this loss; L2 explained why it wants logits, not probabilities.

Gradient with respect to logits

For one-hot target y and softmax probabilities q ,

$$\frac{\partial \text{CE}}{\partial z_k} = q_k - y_k.$$

The gradient has a direct interpretation:

1. lower the logits of over-predicted classes,
2. raise the logit of the under-predicted correct class.

Reverse-mode autodiff in L11 computes this at scale.

The correct class has predicted probability 0.25. What is cross-entropy loss in bits?

A. 0.25

B. 1

C. 2

D. 4

Correct: C — 2 bits

Why: $-\log_2 0.25 = 2$. The loss is the ideal code length assigned to the observed class.

Exponentiate average log loss

Definition

If average cross-entropy is measured in bits per token,

$$\text{perplexity} = 2^{H_{\text{bits}}}.$$

If measured in nats per token,

$$\text{perplexity} = \exp(H_{\text{nats}}).$$

The base must match the logarithm used in the loss.

Interpret uniform predictions

If a model predicts uniformly over K choices,

$$H = \log_2 K \text{ bits.}$$

Then

$$\text{perplexity} = 2^{\log_2 K} = K.$$

Perplexity recovers the number of equally plausible choices.

Non-integer effective choices

If a language model has cross-entropy 1.5 bits/token,

$$\text{perplexity} = 2^{1.5} \approx 2.828.$$

This does not mean exactly 2.828 tokens are possible. It is the equivalent branching factor of a uniform source with the same average log loss.

Sequence cross-entropy

For tokens $x_1, \dots, x_n$,

$$\frac{1}{n} \sum_{t=1}^n -\log_2 q(x_t | x_{<t})$$

is bits per token.

Its exponential is token-level perplexity.

Comparisons require the same tokenization and evaluation data; otherwise the unit “token” has changed.

Read the opening loss curve

The curve from the first slide flattens near 2.2 nats/token.

$$\text{perplexity} = \exp(2.2) \approx 9.0.$$

After training, the model is as uncertain as a uniform choice among about 9 tokens per step; at initialization it was uniform over 50257.

A further drop from 2.2 to 1.5 nats reads as perplexity $9.0 \rightarrow 4.5$: small differences in loss are large differences in behaviour.

Perplexity is not accuracy

Accuracy checks whether the top predicted class is correct.

Perplexity uses the full probability assigned to every observed token.

Two models can have the same top-1 accuracy but different calibration and therefore different cross-entropy or perplexity.

**How much does one variable
reveal about another?**

Dependence as a KL divergence

Mutual information compares the joint distribution with the product of marginals:

$$I(X; Y) = D_{\text{KL}}(p(x, y) \| p(x)p(y)).$$

If X and Y are independent, $p(x, y) = p(x)p(y)$ and $I(X; Y) = 0$.

If observing Y changes predictions about X , mutual information is positive.

L23's noisy-channel limit is this quantity: a channel's capacity is the maximum of $I(\text{input}; \text{output})$ over input distributions.

Entropy reduction

The same quantity can be written

$$I(X; Y) = H(X) - H(X|Y).$$

It is the average reduction in uncertainty about X after observing Y .

Mutual information is symmetric:

$$I(X; Y) = I(Y; X).$$

Two binary examples

Let X be a fair bit.

1. If $Y = X$, then $H(X) = 1$ bit and $H(X|Y) = 0$, so $I(X; Y) = 1$ bit.
2. If Y is an independent fair bit, then $H(X|Y) = H(X) = 1$, so $I(X; Y) = 0$.

One variable either reveals the bit completely or reveals nothing.

The loss used by probabilistic models

One identity joins three modules

$$H(p, q) = H(p) + D_{\text{KL}}(p \| q).$$

1. **Probability:** p describes the data source.
2. **Information:** $-\log q(x)$ is model-assigned code length.
3. **Estimation:** minimizing sample cross-entropy is maximizing likelihood.
4. **Optimization:** gradients change q_θ to reduce mismatch.

The same calculation is read in four different ways.

The L14 loss table, completed

L14 derived the first two rows and deferred the third to this lecture.

You observe	Model	NLL per example	Its famous name
a real y	Gaussian	$(y - \hat{y})^2 / 2\sigma^2 + c$	MSE ✓ L14
yes / no	Bernoulli	$-[y \log p + (1 - y) \log(1 - p)]$	BCE ✓ L14
1 of K classes	Categorical	$-\log q_c$	cross-entropy ✓ today
y with outliers	Laplace	$ y - \hat{y} / b + c$	MAE L14 bonus

Every row is one construction: an observation model's negative log-likelihood — and mean NLL is the cross-entropy $H(\hat{p}, q_\theta)$.

Cross-entropy is the cost of coding data from p with probabilities from q ; KL is the excess cost.

1. $H(p, q) = H(p) + D_{\text{KL}}(p \| q)$.
2. KL is non-negative, asymmetric, and may be infinite.
3. Mean NLL is empirical cross-entropy.
4. Minimizing cross-entropy is maximum likelihood and minimizes forward KL to the empirical distribution.
5. One-hot categorical cross-entropy is $-\log$ probability of the correct class.
6. Perplexity exponentiates average log loss.

Practice

1. Compute $H(p, q)$ for $p = (\frac{1}{2}, \frac{1}{2})$ and $q = (\frac{3}{4}, \frac{1}{4})$ in bits.
2. Use the identity to obtain $D_{\text{KL}}(p\|q)$.
3. A correct class receives probability 0.01. Find its loss in nats and bits.
4. A model reports 2 nats/token. Find perplexity.

1. $H(p, q) = -\frac{1}{2} \log_2\left(\frac{3}{4}\right) - \frac{1}{2} \log_2\left(\frac{1}{4}\right) \approx 1.208$ bits.
2. $H(p) = 1$ bit, so $D_{\text{KL}}(p\|q) \approx 0.208$ bits.
3. $-\ln 0.01 \approx 4.605$ nats; $-\log_2 0.01 \approx 6.644$ bits.
4. $\exp(2) \approx 7.389$.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/info-theory

The **info-theory** article recomputes $H(p)$, $H(p, q)$, and $D_{\text{KL}(p\|q)}$ as you drag probability bars. Set p to today's weather and drag q away from it; the KL readout is the extra cost.

Read before L25 — MacKay Ch. 2.6 (KL divergence) · Ch. 8 (mutual information, skim) · **Assigned:** Olah, *Visual Information Theory* — today's wrong-code story, drawn as pictures. **T12** pairs with this lecture; **A3** (compressor + n-gram LM) goes out.

Next: dependence across time

So far, most examples treated observations as independent. Sequences have structure:

$$p(x_1, \dots, x_n) = p(x_1) \prod_{t=2}^n p(x_t | x_{<t}).$$

A Markov model keeps only the previous state:

$$p(x_t | x_{<t}) \approx p(x_t | x_{t-1}).$$

Next lecture turns this assumption into a transition matrix.

Cross-entropy is the price of believing the wrong distribution.

Next: **Markov Chains** — when the next state depends only on the current one, the whole process is a matrix.